

Studi'OS

Roadmap de refonte UI/UX complète

Objectif : appliquer à toutes les interfaces le langage visuel validé : moderne, clair, accessible, aéré et orienté utilisateur, sans supprimer ni inventer de fonctionnalités. Le dépôt et les contrats/API existants restent la source de vérité fonctionnelle.

PRINCIPE DIRECTEUR

Une page Studi'OS ne doit pas chercher à montrer tout ce qu'elle sait. Elle doit montrer ce dont l'utilisateur a besoin maintenant. L'espace vide est un élément volontaire du design : Overview = résumer, liste = trouver, détail = expliquer, Inspector = exposer la technique.

Cible visuelle

Interface claire sur fond légèrement bleuté, sidebar bleu nuit, cartes blanches peu contrastées, ombres discrètes, grands espacements, typographie lisible, bleu pour les actions, vert pour les états positifs, orange pour l'attente, rouge pour l'erreur et violet pour l'IA. Les écrans doivent respirer : moins de previews simultanées, davantage de navigation vers les pages spécialisées.

Phase	Livrable principal
UI-0	Audit fonctionnel et matrice de migration
UI-1	Design System réutilisable
UI-2	Shell, sidebar, topbar et navigation
UI-3	Accueil aéré
UI-4	Projets + workspace projet
UI-5	Tâches : liste, board, détail
UI-6	Agents IA
UI-7	Bibliothèque
UI-8	Décisions et review
UI-9	Machines
UI-10	Transferts
UI-11	Inspector / Developer
UI-12	Paramètres
UI-13 à 16	États UX, accessibilité, responsive et polish

UI-0 — Audit fonctionnel et gel du périmètre

Avant le redesign, établir précisément ce que l'interface sait déjà faire.

- Inventorier routes, vues, composants, actions utilisateur, permissions, états loading/empty/error et fonctions désactivées.
- Cartographier chaque appel API et les contrats consommés par le frontend.
- Produire la matrice PAGE ACTUELLE -> FONCTIONS -> API -> ACTIONS -> NOUVELLE PAGE -> COMPOSANTS.
- Repérer les éléments purement techniques qui doivent migrer vers Inspector ou un panneau de détails.

GATE : aucune fonctionnalité existante ne disparaît silencieusement.

UI-1 — Design System Studi'OS

Créer les fondations visuelles communes avant de migrer les pages.

- Définir tokens : couleurs, typographie, espacements, rayons, bordures, ombres, breakpoints et z-index.
- Créer les primitives : Button, IconButton, Card, Badge, Input, Select, Tabs, Table, Modal/Drawer, Tooltip, Toast, EmptyState, Skeleton et Status.
- Uniformiser les icônes et les états : action = bleu, succès/online = vert, attente = orange, erreur/bloqué = rouge, IA = violet.
- Garantir des densités cohérentes : compact uniquement dans les listes techniques, confortable par défaut.

GATE : une page de démonstration interne présente tous les composants et leurs principaux états.

UI-2 — App Shell et navigation

Installer le squelette commun de l'application.

- Sidebar : Accueil, Projets, Tâches, Agents IA, Bibliothèque, Décisions, Transferts, Machines, Outils/Inspector, Paramètres.
- Topbar : recherche globale, actions contextuelles limitées, notifications si réellement supportées et compte utilisateur.
- Navigation identique sur toutes les pages ; sidebar compacte sur laptop et drawer sur mobile.
- Ne jamais afficher un élément de navigation fictif uniquement parce qu'il existe dans une maquette.

UI-3 — Accueil

Transformer l'Overview en résumé calme du workspace, pas en cockpit exhaustif.

- Limiter les KPI aux informations immédiatement utiles : projets actifs, tâches en cours, agents, éléments à examiner.
- Afficher seulement quelques projets, activités, tâches et transferts récents, chacun avec une action « Voir tout ».
- État système sous forme de résumé compact ; les diagnostics détaillés restent ailleurs.

- Supprimer les previews redondantes, graphiques décoratifs et répétitions d'information.

GATE : l'écran doit rester compréhensible en quelques secondes et conserver de vrais espaces de respiration.

UI-4 — Projets et workspace projet

Faire du projet le centre de travail contextuel.

- Page Projets : recherche, filtres simples, création et cartes/lignes aérées avec nom, description, statut et quelques indicateurs utiles.
- Page projet : header clair et navigation locale : Vue d'ensemble, Tâches, Activité, Décisions, Agents, Ressources - uniquement si les capacités réelles le permettent.
- Vue d'ensemble projet : état, travail actuel, éléments nécessitant une intervention et activité récente.
- Les informations avancées sont accessibles dans les onglets ou panneaux de détail, pas empilées sur l'overview.

UI-5 — Tâches

Créer une vraie surface de travail quotidienne.

- Deux modes : Liste pour rechercher/trier et Board pour visualiser le flux.
- Board : À faire, En cours, En review, Bloqué, Terminé, selon les statuts réellement supportés.
- Fiche/panneau détail : description, projet, statut, priorité, assignation humain/IA, claims, activité et actions disponibles.
- Le Kanban complet n'apparaît plus sur l'Accueil.

UI-6 — Agents IA

Rendre l'orchestration IA compréhensible sans exposer la plomberie en premier.

- Présenter chaque agent par identité, état, runtime/harness utile, travail actuel et dernière activité.
- Séparer visuellement « qui travaille » des runtimes/providers/bindings qui expliquent comment il fonctionne.
- Fiche agent : capacités, configuration, activité et travaux associés.
- Les IDs et données techniques sont relégués aux détails.

UI-7 — Bibliothèque

Transformer Library en bibliothèque documentaire moderne plutôt qu'en table de base de données.

- Recherche dominante et filtres par types réellement supportés (rules, skills, workflows, agents, models, etc.).
- Entrées centrées sur nom, rôle, type et portée ; éviter de mettre content_schema au premier plan.
- Page de lecture : contenu, métadonnées, versions, compatibilité et actions.
- Section « Détails techniques » repliable pour les champs internes.

UI-8 — Décisions et review

Centraliser ce qui requiert une intervention humaine.

- Présenter clairement la question : « Qu'est-ce qui attend ma décision ? ».
- Catégoriser selon le backend : décisions, AI reviews, conflits, builds, pull requests ou autres éléments réellement disponibles.
- Chaque entrée fournit contexte, raison de l'intervention et actions ; le détail complet vient après ouverture.
- Le compteur « À examiner » de l'Accueil mène ici.

UI-9 — Machines

Donner une lecture instantanée de l'infrastructure disponible.

- Une machine = nom, état de connexion, dernière activité et quelques capacités/indicateurs utiles.
- Associer les transferts et activités pertinentes.
- IDs, payloads et diagnostics avancés restent dans les détails/Inspector.

UI-10 — Transferts

Créer un gestionnaire de transferts simple et lisible.

- Séparer En cours et Récents.
- Afficher source -> destination, nom, taille, progression, état et erreurs de façon explicite.
- Fournir une vue détail pour les diagnostics et métadonnées techniques.

UI-11 — Inspector / Developer

Assumer une surface volontairement plus dense pour le diagnostic.

- Regrouper endpoints, payloads JSON, IDs, événements, diagnostics, informations MCP et état backend.
- Préserver les outils techniques existants tout en les sortant des parcours utilisateur ordinaires.
- Utiliser une présentation adaptée aux développeurs : code, tables compactes, filtres et copie rapide.

UI-12 — Paramètres

Regrouper les réglages dans une surface cohérente.

- Sections possibles : Général, Workspace, IA, Infrastructure, Sécurité, Développeur - seulement si elles correspondent aux réglages réels.
- Formulaire aérés, descriptions courtes et confirmation des actions destructives.
- Éviter les pages de configuration composées d'un simple dump de champs.

UI-13 — États UX globaux

Uniformiser le comportement de l'application.

- Loading : skeletons légers ; Empty : explication + action ; Error : message humain + détail facultatif.
- Prévoir offline, permission denied, données partielles et confirmations destructives.
- Interdire les écrans qui se contentent d'afficher « No data » sans contexte.

UI-14 — Accessibilité

Faire de la lisibilité une exigence fonctionnelle.

- Contrastes WCAG AA, focus visible, navigation clavier, cibles suffisamment grandes et labels de formulaires.
- Ne jamais communiquer un état uniquement par couleur.
- ARIA lorsque nécessaire, zoom navigateur correct et prefers-reduced-motion.

UI-15 — Responsive

Adapter la hiérarchie plutôt que réduire mécaniquement le desktop.

- Concevoir et vérifier : desktop -> laptop -> tablette -> mobile.
- Sur petit écran : cartes empilées, sidebar en drawer, actions secondaires dans un menu et détails en plein écran.
- Tables scrollables ou transformées lorsque nécessaire ; ne pas conserver les dashboards multi-colonnes sur mobile.

UI-16 — Polish, nettoyage et validation finale

Faire la passe de cohérence uniquement après migration de toutes les pages.

- Audit visuel page par page : espacements, typographie, icônes, boutons, badges, transitions, modales et tooltips.
- Supprimer CSS mort, composants obsolètes et styles parallèles.
- Vérifier responsive, accessibilité, régressions fonctionnelles et cohérence des contrats API.
- Comparer l'ensemble au langage visuel de référence : moderne, digeste, accessible et volontairement aéré.

GATE : toutes les pages sont migrées ; aucune ancienne interface ne subsiste sans justification documentée.

Règles d'exécution pour l'agent

1. Le dépôt est la source de vérité fonctionnelle. Les maquettes donnent la direction UX/UI mais ne justifient jamais l'invention d'un endpoint ou d'une fonctionnalité.
2. Préserver les contrats et comportements existants sauf correction explicitement justifiée et testée.
3. Ne pas traiter toute la roadmap en un énorme changement opaque : avancer phase par phase avec vérifications et commits cohérents.
4. Réutiliser le Design System ; éviter le CSS ad hoc page par page.
5. Ne pas densifier un écran pour « utiliser l'espace ». L'espace vide est intentionnel.
6. Les détails techniques doivent être accessibles, mais rarement dominants hors Inspector.
7. Tester chaque page dans ses états normal, vide, chargement, erreur et responsive pertinents.
8. Documenter les arbitrages qui modifient l'architecture frontend ou l'expérience utilisateur.

DÉFINITION DE TERMINÉ

La refonte n'est terminée que lorsque toutes les pages existantes ont été migrées vers le nouveau langage visuel, que les parcours restent fonctionnels, que l'accessibilité et le responsive sont vérifiés, et qu'aucune fonctionnalité backend existante n'a été perdue.